

CMPEN 331 Final Project Report

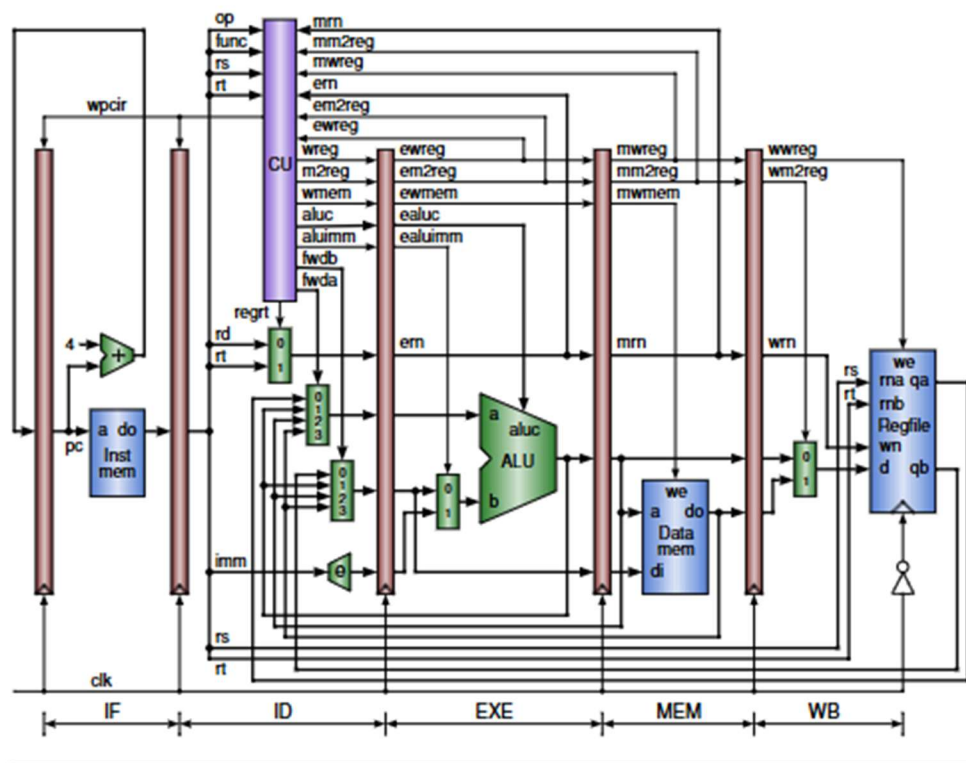
Ryan Cao

rjc6651@psu.edu

Abstract

The goal of this project was to implement a CPU over an FPGA using Verilog. This was done using the MIPS architecture. The CPU data path was pipelined into five stages: instruction fetch, instruction decode, execute, memory, and write back. Forwarding from ALU to ALU and MEM to ALU was also implemented to handle data hazards in order to reduce the number of stalls needed.

Introduction



This is the diagram of the data path implemented in Verilog. The instruction fetch stage is where the program counter is used to retrieve an instruction from instruction memory. This instruction is identified by bitfields in the instruction decode stage depending on its instruction type. This stage is also where the CPU determines whether forwarding is possible/necessary. This is done in the control unit where signals from later stages from previous instructions are sent back to be compared with source registers rs and rt, producing the fwda and fwdb signals that are fed into two multiplexers. These multiplexers

then choose from four signals from later stages. The results from forwarding, the control unit, and sign extension are fed into the execution stage where the ALU performs operations specified by the instruction opcode and func fields. The results from the EXE stage are then fed into the memory stage where data can be read from memory or written to memory depending on the instruction type. Then the final stage is the writeback stage where data is written to the register file depending on the destination register. Between each stage there are registers designated specifically to hold the results from its respective previous stage. These physically separate the data path so that pipelining and forwarding is possible. Without these registers, data would be lost before it can be used. They also allow the data path to run on a clock, which helps coordinate data flow.

Design Code

```
1 `timescale 1ns / 1ps
2 //////////////////////////////////////
3 // Company: Penn State
4 // Engineer: Ryan Cao
5 //
6 // Create Date: 3/22/2026
7 // Design Name: Lab 3 - Instruction Fetch and Decode
8 // Module Name: datapath
9 // Project Name: Lab 3 - Instruction Fetch and Decode
10 // Target Devices: XC7Z010-CLG400-1
11 // Description: Implementation of IF and ID stages of a 5-stage pipelined CPU
12 //
13 // Instructions to implement:
14 //   Address 100: lw $v0, 00($at)  -> lw $2, 0($1)
15 //   Address 104: lw $v1, 04($at)  -> lw $3, 4($1)
16 //   Assume register $at ($1) has value 0
17 //
18 //////////////////////////////////////
19
20 //=====
21 // TOP-LEVEL DATAPATH MODULE
22 //=====
23 // This module connects all the pipeline stages together.
24 // Input:  clk - clock signal
25 // Output: None (all signals are internal)
26 //
27 // You need to:
28 // 1. Declare all internal wires to connect the modules
29 // 2. Instantiate all 9 modules and connect them properly
30 // 3. Decode the instruction fields (op, rs, rt, rd, func, imm) from dinstOut
31 //=====
32
33 module Datapath(
34     input clk,
35     output [31:0] wbData
36 );
37
```

```

64   ):
65
66   // TODO: Declare wires for IF stage
67   // Hint: You need wires for pc, nextpc, instOut
68   wire [31:0] pc, nextpc, instOut;
69
70   // TODO: Declare wire for IF/ID pipeline register output
71   wire [31:0] dinstOut;
72   // Hint: dinstOut holds the instruction in the ID stage
73
74   // TODO: Decode instruction fields from dinstOut
75   // Hint: Use assign statements to extract:
76   // Example: op = dinstOut[31:26] (6 bits - opcode)
77   wire [5:0] op, func;
78   wire [4:0] rs, rt, rd;
79   wire [15:0] imm;
80   assign op = dinstOut[31:26];
81   assign rs = dinstOut[25:21];
82   assign rt = dinstOut[20:16];
83   assign rd = dinstOut[15:11];
84   assign shamt = dinstOut[10:6];
85   assign func = dinstOut[5:0];
86   assign imm = dinstOut[15:0];
87
88
89   // TODO: Declare wires for control unit outputs
90   // Hint: wreg, m2reg, wmem, aluc[3:0], aluimm, regrt;
91   wire wreg, m2reg, wmem, aluimm, regrt;
92   wire [3:0] aluc;
93   wire [1:0] fwda, fwdb;
94   wire [31:0] fwva, fwvb;
95   // TODO: Declare wires for ID stage
96   // Hint: destReg[4:0], qa[31:0], qb[31:0], imm32[31:0]
97   wire [4:0] destReg;
98   wire [31:0] qa, qb, imm32;
99   // TODO: Declare wires for ID/EXE pipeline register outputs
100  // Hint: Use 'e' prefix for EXE stage signals (ewreg, em2reg, etc.)
101
102  wire ewreg, em2reg, ewmem, ealuimm;
103  wire [3:0] ealuc;
104  wire [4:0] edestReg;
105  wire [31:0] eqa, eqb, eimm32;
106
107  // ALUMux output wire
108  wire [31:0] b;
109
110  // wires for EXE/MEM stage
111  wire mwreg, mm2reg, mmemem;
112  wire [4:0] mdestReg;
113  wire [31:0] r, mr, mqb, mdo;
114
115  // Wires for MEM/WB stage
116  wire wwreg, wm2reg;
117  wire [4:0] wdestReg;
118  wire [31:0] wr, wdo, wbData;
119
120  //=====
121  // STAGE 1: INSTRUCTION FETCH (IF) - Instantiate modules
122  //=====
123
124  // TODO: Instantiate PC (Program Counter)
125  PC pc1(clk, nextpc, pc);
126  // TODO: Instantiate PCAdd4 (PC + 4 Adder)
127  PCAdd4 pcadd4(pc, nextpc);
128  // TODO: Instantiate IM (Instruction Memory)

```

```

126 // TODO: Instantiate PCAdd4 (PC + 4 Adder)
127 PCAdd4 pcadd4 (pc, nextpc);
128 // TODO: Instantiate IM (Instruction Memory)
129 IM iml(pc,instOut);
130 // TODO: Instantiate IFID (IF/ID Pipeline Register)
131 IFID ifid(clk, instOut, dinstOut);
132 //=====
133 // STAGE 2: INSTRUCTION DECODE (ID) - Instantiate modules
134 //=====
135
136 // TODO: Instantiate CU (Control Unit)
137 CU cu(op, func, rs, rt, mdestReg, mm2reg, mwreg, edestReg, em2reg, ewreg, wreg, m2reg, wmem, aluc, aluimm, regrt, fwdB, fwdA);
138 // TODO: Instantiate Regfile (Register File)
139 Regfile rfile(rs, rt, wdestReg, wbData, wwreg, clk, qa, qb);
140 // TODO: Instantiate Mux (Regrt Multiplexer)
141 Mux mux(rd, rt, regrt, destReg);
142 // TODO: Instantiate Ext (Sign Extender)
143
144 // forwarding muxes
145 FWDmux fwdamux(fwda, qa, r, mr, mdo, fwva);
146 FWDmux fwdbmux(fwdb, qb, r, mr, mdo, fwvb);
147
148 Ext ext(imm, imm32);
149 // TODO: Instantiate IDEXE (ID/EXE Pipeline Register)
150 IDEXE idexe(clk, wreg, m2reg, wmem, aluc, aluimm, destReg, fwva, fwvb, imm32, ewreg, em2reg, ewmem, ealuc, ealuimm, edestReg, eqa, eqb, eimm32);
151 //=====
152 // STAGE 3: INSTRUCTION EXECUTE (EXE) - Instantiate modules
153 //=====
154 // Implement ALUMux, ALU, EXEMEM
155 ALUMux alumux(ealuimm, eqb, eimm32, b);
156 ALU alu(b, eqa, ealuc, r);
157 EXEMEM exemem (clk, ewreg, em2reg, ewmem, edestReg, eqb, r, mwreg, mm2reg, mwmem, mdestReg, mr, mqb);
158
159 //=====
160 // STAGE 4: MEMORY (MEM) - Instantiate modules
161 //=====
162 DM datamem(clk, mr, mqb, mmem, mdo);
163 MEMWB memwb(clk, mwreg, mm2reg, mdestReg, mr, mdo, wwreg, wm2reg, wdestReg, wr, wdo);
164
165 //=====
166 // STAGE 5: WRITEBACK (WB) - Instantiate modules
167 //=====
168 WbMux wbmux(wr, wdo, wm2reg, wbData);
169
170 endmodule
171
172 //=====
173 // STAGE 1: INSTRUCTION FETCH (IF) MODULES
174 //=====
175
176 //-----
177 // PC - Program Counter
178 //-----
179 // Holds the address of the current instruction being fetched.
180 // Starts at address 100 as specified in the lab.
181 //
182 // Inputs:
183 //   clk      - clock signal (update PC on positive edge)
184 //   nextpc   - the next PC value (PC + 4)
185 //
186 // Outputs:
187 //   pc       - current program counter value (32 bits)
188 //
189 //
190

```

```

188 // pc - current program counter value (32 bits)
189 //
190 // Hints:
191 // - Use an initial block to set pc = 100
192 // - Use always @(posedge clk) to update pc with nextpc
193 // - Use non-blocking assignment (<=) in sequential logic
194 //-----
195
196 module PC(
197     input clk,
198     input [31:0] nextpc,
199     output reg [31:0] pc
200 );
201 // TODO: Implement the program counter
202 // Initialize pc to 100, update to nextpc on each clock edge
203 initial begin
204     pc <= 100;
205 end
206
207 always@(posedge clk) begin
208     pc <= nextpc;
209 end
210 endmodule
211
212 //-----
213 // PCAdd4 - PC + 4 Adder
214 //-----
215 // Calculates the address of the next sequential instruction.
216 //
217 // Inputs:
218 // pc - current program counter value (32 bits)
219 //
220 // Outputs:
221 // nextpc - next program counter value (pc + 4) (32 bits)
222 //
223 // Hints:
224 // - This is combinational logic, use always @(*)
225 //-----
226
227 module PCAdd4(
228     input [31:0] pc,
229     output reg [31:0] nextpc
230 );
231 // TODO: Implement the PC + 4 adder
232 always@(*) begin
233     nextpc <= pc + 4;
234 end
235 endmodule
236
237 //-----
238 // IM - Instruction Memory
239 //-----
240 // Stores the program instructions. Read-only memory.
241 // Address 100 (index 25) and 104 (index 26) contain the lv instructions.
242 //
243 // Inputs:
244 // pc - program counter (address to read from) (32 bits)
245 //
246 // Outputs:
247 // instOut - instruction at the given address (32 bits)
248 //
249 // Hints:
250 //
251 //

```

```

251 // Hints:
252 // - Declare a reg array: reg [31:0] instructions [0:127]
253 // - Use initial block
254 //-----
255
256 module IM(
257     input [31:0] pc,
258     output reg [31:0] instOut
259 );
260 // TODO: Declare instruction memory array
261 reg [31:0] instructions [0:127];
262
263 // TODO: Initialize instructions at index 25 and 26
264 initial begin
265     instructions[25] = 32'b00000000001000100000100000100000; // add $3, $1, $2
266     instructions[26] = 32'b00000001001000110010000000100010; // sub $4, $9, $3
267     instructions[27] = 32'b000000000110100100100000100101; // or $5, $3, $9
268     instructions[28] = 32'b00000000011010010011000000100110; // xor $6, $3, $9
269     instructions[29] = 32'b00000000011010010011000000100100; // and $7, $3, $9
270 end
271
272 // TODO: Read instruction based on pc (use pc >> 2 or pc / 4 as index)
273 always @(*)begin
274     instOut <= instructions[pc >> 2];
275 end
276 endmodule
277
278 //-----
279 // IFID - IF/ID Pipeline Register
280 //-----
281 // Stores the instruction fetched in IF stage for use in ID stage.
282 // Updates on the positive edge of the clock.
283 //
284 // Inputs:
285 //   clk - clock signal
286 //   instOut - instruction from instruction memory (32 bits)
287 //
288 // Outputs:
289 //   dinstOut - instruction passed to ID stage (32 bits)
290 //
291 // Hints:
292 // - Use always @(posedge clk)
293 // - Use non-blocking assignment (<=)
294 //-----
295
296 module IFID(
297     input clk,
298     input [31:0] instOut,
299     output reg [31:0] dinstOut
300 );
301 // TODO: Implement the IF/ID pipeline register
302 always @(posedge clk) begin
303     dinstOut <= instOut;
304 end
305 endmodule
306
307 //=====
308 // STAGE 2: INSTRUCTION DECODE (ID) MODULES
309 //=====
310
311 //-----
312
313 // CP - Control Unit

```

```

313 //-----
314 // CU - Control Unit
315 //-----
316 // Generates control signals based on the opcode and function code.
317 //
318 // Inputs:
319 //   op      - opcode field from instruction (6 bits)
320 //   func    - function field from instruction (6 bits)
321 //
322 // Outputs:
323 //   wreg    - write enable for register file (1 bit)
324 //   m2reg   - memory to register (1 = load data from memory) (1 bit)
325 //   wmem    - write enable for data memory (1 bit)
326 //   aluc    - ALU control signals (4 bits)
327 //   aluimm  - ALU source B select (1 = use immediate) (1 bit)
328 //   regrt   - register destination select (0 = rd, 1 = rt) (1 bit)
329 //
330 // Hints:
331 //   - Use a case statement on the opcode
332 //   - Add a default case to handle undefined opcodes
333 //-----
334
335 module CU(
336     input [5:0] op,
337     input [5:0] func,
338     input [4:0] rs,
339     input [4:0] rt,
340     input [4:0] mdestReg,
341     input m2reg,
342     input mwreg,
343     input [4:0] edestReg,
344     input em2reg,
345     input ewreg,
346     output reg wreg,
347     output reg m2reg,
348     output reg wmem,
349     output reg [3:0] aluc,
350     output reg aluimm,
351     output reg regrt,
352     output reg [1:0] fwdb,
353     output reg [1:0] fwda
354 );
355 // TODO: Implement the control unit using a case statement
356 always@(*)begin
357
358     case(op)
359         // Load Word opcode
360         6'b100011: begin
361             regrt = 1;
362             aluimm = 1;
363             m2reg = 1;
364             wmem = 0;
365             wreg = 1;
366             aluc = 4'b0010;
367         end
368
369         //Store Word opcode
370         6'b101011: begin
371             aluimm = 1;
372             wreg = 0;
373             wmem = 1;
374             aluc = 4'b0010;
375         end

```

```

375 end
376
377 //R-type instruction opcode
378 6'b000000: begin
379     regrt = 0;
380     aluimm = 0;
381     m2reg = 0;
382     wreg = 1;
383     wmem = 0;
384
385     // Analyzing function bitfield
386     case(func)
387         6'b100000: aluc = 4'b0010; // add
388         6'b100010: aluc = 4'b0110; // subtract
389         6'b100100: aluc = 4'b0000; // AND
390         6'b100101: aluc = 4'b0001; // OR
391         6'b100110: aluc = 4'b0011; // XOR
392         6'b101010: aluc = 4'b0111; // slt
393     endcase
394 end
395
396 //beq instruction opcode
397 6'b000100: begin
398     aluc = 0;
399     wreg = 0;
400     wmem = 0;
401     aluc = 4'b0110;
402 end
403
404 default: begin
405     wreg = 1;
406     m2reg = 0;
407     wmem = 0;
408     aluimm = 0;
409     regrt = 0;
410     aluc = 4'b0010;
411 end
412 endcase
413
414 // forwarding logic
415 if (rs == mdestReg && m2reg == 1) fwda = 2'b11; // load word dependency
416 else if (rs == mdestReg) fwda = 2'b10; // MEM to ALU forwarding
417 else if (rs == edestReg) fwda = 2'b01; // ALU to ALU forwarding
418 else fwda = 2'b00; // no hazards
419
420 if (rt == mdestReg && m2reg == 1) fwdb = 2'b11; // load word dependency
421 else if (rt == mdestReg) fwdb = 2'b10; // MEM to ALU forwarding
422 else if (rt == edestReg) fwdb = 2'b01; // ALU to ALU forwarding
423 else fwdb = 2'b00; // no hazards
424 end
425
426 endmodule
427
428
429 //-----
430 // Regfile - Register File
431 //-----
432 // Contains 32 general-purpose registers. Provides two read ports.
433 // All registers should be initialized to 0.
434 //
435 // Inputs:
436 //   rs - source register 1 address (5 bits)
437 //   rt - source register 2 address (5 bits)
438

```

```

439 // Outputs:
440 //   qa - data from register rs (32 bits)
441 //   qb - data from register rt (32 bits)
442 //
443 // Hints:
444 //   - Declare a reg array: reg [31:0] registers [0:31]
445 //   - Use a for loop in initial block to set all registers to 0
446 //   - Use always @(*) for combinational read logic
447 //-----
448
449 module Regfile(
450     input [4:0] rs,
451     input [4:0] rt,
452     input [4:0] wdestReg,
453     input [31:0] wbData,
454     input wwreg,
455     input clk,
456     output reg [31:0] qa,
457     output reg [31:0] qb
458 );
459     // TODO: Declare register array
460     reg [31:0] registers [0:31];
461
462     integer i;
463
464     // initialize register words
465     initial begin
466         registers[0] = 32'h00000000;
467         registers[1] = 32'hA00000AA;
468         registers[2] = 32'h10000011;
469         registers[3] = 32'h20000022;
470         registers[4] = 32'h30000033;
471         registers[5] = 32'h40000044;
472         registers[6] = 32'h50000055;
473         registers[7] = 32'h60000066;
474         registers[8] = 32'h70000077;
475         registers[9] = 32'h80000088;
476         registers[10] = 32'h90000099;
477
478         for ( i = 11; i < 32; i = i + 1) begin
479             registers[i] = 0;
480         end
481     end
482
483     always@(negedge clk) begin
484         if (wwreg == 1) registers[wdestReg] <= wbData;
485     end
486
487     // TODO: Read registers based on rs and rt
488     always@(*) begin
489         qa = registers[rs];
490         qb = registers[rt];
491     end
492 endmodule
493
494 //-----
495 // Mux - Regrt Multiplexer
496 //-----
497 // Selects the destination register number.
498 // For R-type instructions, destination is rd.
499 // For I-type instructions (like lw), destination is rt.
500 //
501 // Inputs:
502 //   rd - destination register number for R-type instructions
503

```

```

504 // rt - destination register for I-type (5 bits)
505 // regrt - select signal (0 = rd, 1 = rt)
506 //
507 // Outputs:
508 // destReg - selected destination register (5 bits)
509 //
510 // Hints:
511 // - Use if-else
512 // - When regrt = 0, select rd
513 // - When regrt = 1, select rt
514 //-----
515
516 module Mux(
517     input [4:0] rd,
518     input [4:0] rt,
519     input regrt,
520     output reg [4:0] destReg
521 );
522 // TODO: Implement the multiplexer
523 always@(*) begin
524     if(regrt == 1)
525         destReg = rt;
526     else
527         destReg = rd;
528     end
529
530 endmodule
531
532 // forwarding mux in the ID stage
533 // regdata is either qa or qb
534 module FWDmux(
535     input [1:0] sel,
536     input [31:0] regdata,
537     input [31:0] r,
538     input [31:0] mr,
539     input [31:0] mdo,
540     output reg [31:0] result
541 );
542
543 always@(*) begin
544     case (sel)
545         0: result = regdata;
546         1: result = r;
547         2: result = mr;
548         3: result = mdo;
549     endcase
550 end
551 endmodule
552
553 //-----
554 // Ext - Immediate Extender
555 //-----
556 // Extends the 16-bit immediate value to 32 bits using sign extension.
557 //
558 // Inputs:
559 // imm - 16-bit immediate value from instruction
560 //
561 // Outputs:
562 // imm32 - 32-bit sign-extended immediate
563 //
564 // Hints:
565 // - Use concatenation with replication
566 // - imm[15] is the sign bit
567 // - Replicate the sign bit 16 times and concatenate with original imm
568

```

```

566 : // - Use concatenation with replication
567 : // - imm[15] is the sign bit
568 : // - Replicate the sign bit 16 times and concatenate with original imm
569 : //-----
570 :
571 : module Ext(
572 :     input [15:0] imm,
573 :     output reg [31:0] imm32
574 : );
575 :     // TODO: Implement sign extension
576 :     always@(*)begin
577 :         imm32 <= ({16{imm[15]}}, imm);
578 :     end
579 : endmodule
580 :
581 :
582 : //-----
583 : // IDEXE - ID/EXE Pipeline Register
584 : //-----
585 : // Stores all control signals and data from ID stage for use in EXE stage.
586 : // Updates on the positive edge of the clock.
587 : //
588 : // Inputs:
589 : // clk - clock signal
590 : // wreg - write register enable (1 bit)
591 : // m2reg - memory to register (1 bit)
592 : // wmem - write memory enable (1 bit)
593 : // aluc - ALU control (4 bits)
594 : // aluimm - ALU immediate select (1 bit)
595 : // destReg - destination register (5 bits)
596 : // qa - register data A (32 bits)
597 : // qb - register data B (32 bits)
598 : // imm32 - sign-extended immediate (32 bits)
599 : //
600 : // Outputs (active in EXE stage, prefix 'e'):
601 : // ewreg - write register enable
602 : // em2reg - memory to register
603 : // ewmem - write memory enable
604 : // ealuc - ALU control
605 : // ealuimm - ALU immediate select
606 : // edestReg - destination register
607 : // eqa - register data A
608 : // eqb - register data B
609 : // eimm32 - sign-extended immediate
610 : //
611 : // Hints:
612 : // - Use always @(posedge clk)
613 : // - Use non-blocking assignments (<=)
614 : // - Transfer each input to its corresponding output
615 : //-----
616 :
617 : module IDEXE(
618 :     input clk,
619 :     input wreg,
620 :     input m2reg,
621 :     input wmem,
622 :     input [3:0] aluc,
623 :     input aluimm,
624 :     input [4:0] destReg,
625 :     input [31:0] qa,
626 :     input [31:0] qb,
627 :     input [31:0] imm32,
628 :     output reg ewreg,
629 :     output reg em2reg,
630 :     output reg ewmem,

```

```

628     output reg ewreg,
629     output reg em2reg,
630     output reg ewmem,
631     output reg [3:0] ealuc,
632     output reg ealuimm,
633     output reg [4:0] edestReg,
634     output reg [31:0] eqa,
635     output reg [31:0] eqb,
636     output reg [31:0] eimm32
637 );
638 // TODO: Implement the ID/EXE pipeline register
639 always@(posedge clk) begin
640     ewreg <= wreg;
641     em2reg <= m2reg;
642     ewmem <= wmem;
643     ealuc <= aluc;
644     ealuimm <= aluimm;
645     edestReg <= destReg;
646     eqa <= qa;
647     eqb <= qb;
648     eimm32 <= imm32;
649 end
650
651 endmodule
652
653 // ALUMux selects between eqb and eimm32 according to ealuimm
654 module ALUMux(
655     input ealuimm,
656     input [31:0] eqb,
657     input [31:0] eimm32,
658     output reg [31:0] b
659 );
660
661 always@(*) begin
662     if (ealuimm == 0) b <= eqb;
663     else b <= eimm32;
664 end
665 endmodule
666
667
668
669 // ALU operates on b and eqa depending on ealuc, outputting to r
670 module ALU(
671     input [31:0] b,
672     input [31:0] eqa,
673     input [3:0] ealuc,
674     output reg [31:0] r
675 );
676
677 always@(*) begin
678     case(ealuc)
679         // add
680         4'b0010: r <= b + eqa;
681
682         // subtract
683         4'b0110: r <= eqa - b;
684
685         // AND
686         4'b0000: r <= b & eqa;
687
688         // OR
689         4'b0001: r <= b | eqa;
690
691         // XOR
692         4'b0011: r <= b ^ eqa;

```

```

691 // XOR
692 4'b0011: z <= b ^ eqa;
693
694     endcase
695 end
696
697
698 endmodule;
699
700
701
702 // pipeline register EXE/MEM
703 module EXEMEM(
704     input clk,
705     input ewreg,
706     input em2reg,
707     input ewmem,
708     input [4:0] edestReg,
709     input [31:0] eqb,
710     input [31:0] z,
711     output reg mwreg,
712     output reg mm2reg,
713     output reg mwmem,
714     output reg [4:0] mdestReg,
715     output reg [31:0] mr,
716     output reg [31:0] mqb
717 );
718
719 always@(posedge clk) begin
720     mwreg <= ewreg;
721     mm2reg <= em2reg;
722     mwmem <= ewmem;
723     mdestReg <= edestReg;
724     mr <= z;
725     mqb <= eqb;
726 end
727 endmodule
728
729
730
731 module DM(
732     input clk,
733     input [31:0] mr,
734     input [31:0] mqb,
735     input mwmem,
736     output reg [31:0] mdo
737 );
738
739     reg [31:0] dm [0:63];
740
741 // Initializing first 10 words in data memory
742 initial begin
743     dm[0] = 32'hA00000AA;
744     dm[1] = 32'h10000011;
745     dm[2] = 32'h20000022;
746     dm[3] = 32'h30000033;
747     dm[4] = 32'h40000044;
748     dm[5] = 32'h50000055;
749     dm[6] = 32'h60000066;
750     dm[7] = 32'h70000077;
751     dm[8] = 32'h80000088;
752     dm[9] = 32'h90000099;
753 end

```

```

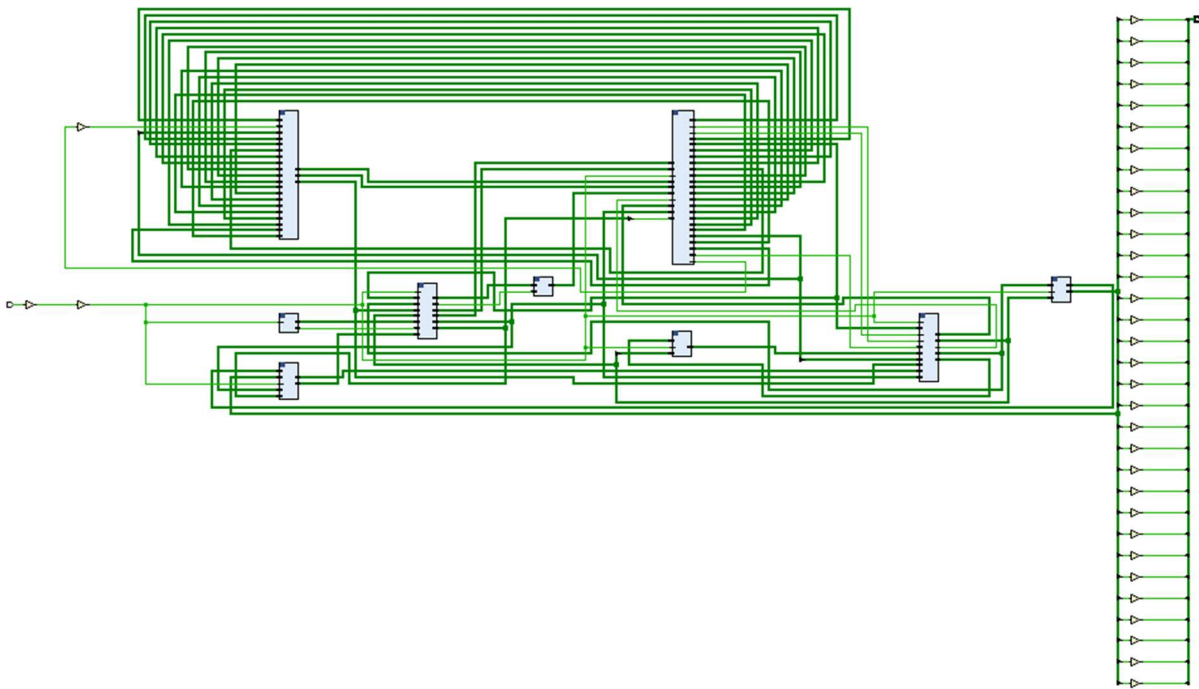
754 :
755 : // reading from data memory on any edge
756 : ○ always@(*) begin
757 : ○   mdo = dm[mr >> 2];
758 : ○ end
759 :
760 : // writing to data memory on negedge
761 : ○ always@(negedge clk) begin
762 : ○   if (mmem == 1) begin
763 : ○     dm[mr >> 2] <= mqb;
764 : ○   end
765 : ○ end
766 : ○ endmodule;
767 :
768 : module MEMWB(
769 :   input clk,
770 :   input mwreg,
771 :   input mm2reg,
772 :   input [4:0] mdestReg,
773 :   input [31:0] mr,
774 :   input [31:0] mdo,
775 :   output reg wwreg,
776 :   output reg wm2reg,
777 :   output reg [4:0] wdestReg,
778 :   output reg [31:0] wr,
779 :   output reg [31:0] wdo
780 : );
781 :
782 : ○ always@(posedge clk) begin
783 : ○   wwreg <= mwreg;
784 : ○   wm2reg <= mm2reg;
785 : ○   wdestReg <= mdestReg;
786 : ○   wr <= mr;
787 : ○   wdo <= mdo;
788 : ○ end
789 : endmodule
790 :
791 :
792 : module WbMux(
793 :   input [31:0] wr,
794 :   input [31:0] wdo,
795 :   input wm2reg,
796 :   output reg [31:0] wbData
797 : );
798 :
799 : ○ always@(*) begin
800 : ○   if (wm2reg==0) wbData = wr;
801 : ○   else wbData = wdo;
802 : ○ end
803 : endmodule

```

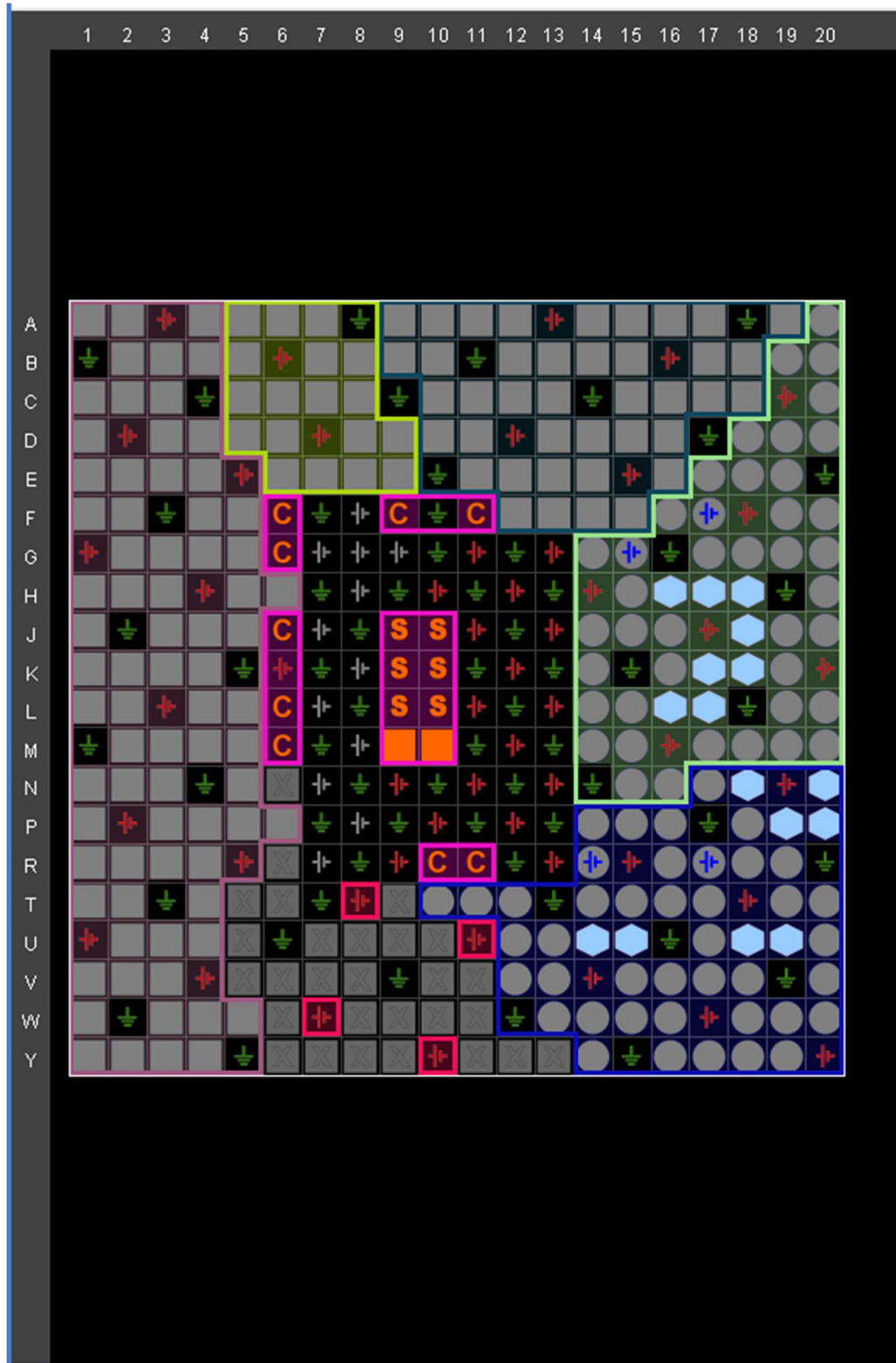
Test Bench

```
1 // timescale 1ns / 1ps
2 ///////////////////////////////////////////////////////////////////
3 // Company: Penn State
4 // Engineer: Ryan Cao
5 //
6 // Create Date: 3/22/2024
7 // Design Name: Lab 3 - Instruction Fetch and Decode Testbench
8 // Module Name: testbench
9 // Project Name: Lab 3 - Instruction Fetch and Decode
10 // Target Device: AC1015-1000-1
11 // Description: Testbench for Lab 3 - IF and ID stages of pipelined CPU
12 //
13 // This testbench verifies the IF and ID stages by:
14 // 1. Generating a clock signal
15 // 2. Running the simulation for multiple clock cycles
16 // 3. Observing the signals in the waveform viewer
17 //
18 ///////////////////////////////////////////////////////////////////
19
20 // TESTBENCH MODULE
21
22 // The testbench instantiates the datapath and provides the clock signal.
23 // All internal signals are wired inside the datapath module.
24 //
25 // To view internal signals in simulation:
26 // 1. In Verilog, run behavioral simulation
27 // 2. In the waveform viewer, expand the datapath instance (dut)
28 // 3. Add the signals you want to observe:
29 //    - IF stages: pc, swpcsr, instchr
30 //    - IF/ID registers: aluinstbus
31 //    - ID stages: op, sw, sh, shd, fncr, imm
32 //    - Control signals: vreg, m2reg, vmm, alub, aluimm, rregt
33 //    - Register file outputs: ra, qb
34 //    - Sign extension output: smul2
35 //    - ID/EXN register outputs: wreg, m2reg, vmm, valub, valuimm,
36 //      subfides, vq, vq, smul2
37 //
38 // Expected behavior:
39 // Cycle 1: PC=000, fetch 1st 32, 5(1)
40 // Cycle 2: PC=004, fetch 1st 32, 4(1), decode 1st 32, 5(1)
41 // Cycle 3: PC=008, decode 1st 32, 4(1), execute 1st 32, 5(1)
42 // ...
43 ///////////////////////////////////////////////////////////////////
44
45 module testbench()
46 //-----
47 // Signal Declaration
48 //-----
49 // TODO: Declare a reg for the clock signal
50 // Hint: reg clk;
51
52 reg clk;
53 wire [31:0] wData;
54
55 //-----
56 // Device Under Test (DUT) Instantiation
57 //-----
58 // TODO: Instantiate the datapath module
59 // Hint: The datapath only has clk as input
60
61 Datapath dp(
62     .clk(clk),
63
64     .datapath_op1
65     .clk(clk),
66     .wData(wData)
67 );
68
69 //-----
70 // Clock Generation
71 //-----
72 // TODO: Initialize the clock to 0
73 // Hint: Use an initial block
74 initial begin
75     clk = 0;
76 end
77
78 // TODO: Generate a clock with 10ns period (0ns high, 5ns low)
79 // Hint: Use an always block with #5 delay
80
81 always #5 clk = ~clk;
82 endmodule
83
```

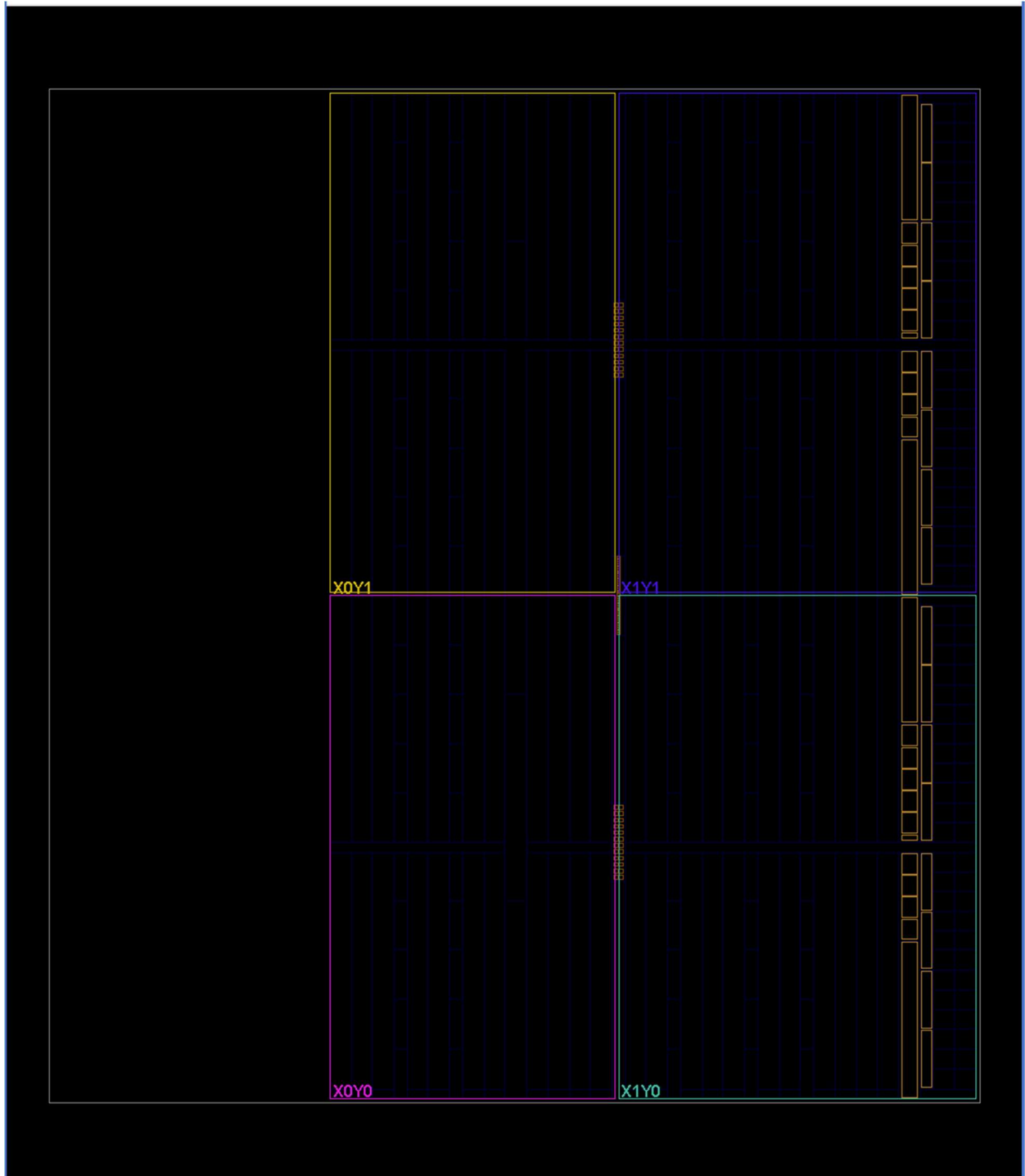
Schematics



IO Planning



Floor Planning



Bitstream

```
1  set_property PACKAGE_PIN A20 [get_ports {wbData[31]}]
2  set_property PACKAGE_PIN B19 [get_ports {wbData[30]}]
3  set_property PACKAGE_PIN B20 [get_ports {wbData[29]}]
4  set_property PACKAGE_PIN C20 [get_ports {wbData[28]}]
5  set_property PACKAGE_PIN D18 [get_ports {wbData[27]}]
6  set_property PACKAGE_PIN D19 [get_ports {wbData[26]}]
7  set_property PACKAGE_PIN D20 [get_ports {wbData[25]}]
8  set_property PACKAGE_PIN E17 [get_ports {wbData[24]}]
9  set_property PACKAGE_PIN E18 [get_ports {wbData[23]}]
10 set_property PACKAGE_PIN E19 [get_ports {wbData[22]}]
11 set_property PACKAGE_PIN F16 [get_ports {wbData[21]}]
12 set_property PACKAGE_PIN F19 [get_ports {wbData[19]}]
13 set_property PACKAGE_PIN F20 [get_ports {wbData[18]}]
14 set_property PACKAGE_PIN G17 [get_ports {wbData[15]}]
15 set_property PACKAGE_PIN G18 [get_ports {wbData[14]}]
16 set_property PACKAGE_PIN G19 [get_ports {wbData[13]}]
17 set_property PACKAGE_PIN G20 [get_ports {wbData[12]}]
18 set_property PACKAGE_PIN H15 [get_ports {wbData[11]}]
19 set_property PACKAGE_PIN H20 [get_ports {wbData[7]}]
20 set_property PACKAGE_PIN J14 [get_ports {wbData[6]}]
21 set_property PACKAGE_PIN J15 [get_ports {wbData[5]}]
22 set_property PACKAGE_PIN J16 [get_ports {wbData[4]}]
23 set_property PACKAGE_PIN J19 [get_ports {wbData[2]}]
24 set_property PACKAGE_PIN J20 [get_ports {wbData[1]}]
25 set_property PACKAGE_PIN K14 [get_ports {wbData[20]}]
26 set_property PACKAGE_PIN K16 [get_ports {wbData[10]}]
27 set_property PACKAGE_PIN K19 [get_ports {wbData[9]}]
28 set_property PACKAGE_PIN L14 [get_ports {wbData[8]}]
29 set_property PACKAGE_PIN L19 [get_ports {wbData[3]}]
30 set_property PACKAGE_PIN L15 [get_ports {wbData[16]}]
31 set_property PACKAGE_PIN H16 [get_ports clk]
32
33 set_property PACKAGE_PIN G14 [get_ports {wbData[17]}]
34 set_property PACKAGE_PIN L20 [get_ports {wbData[0]}]
35
```